

Architecture & Operations Proposal

Technical AI Assessment — Senior Site Reliability Engineer, MetaCTO · Guilherme Giusti · 2026-06-25 · Global VOD streaming on AWS

A global **video-on-demand (VOD)** streaming platform (Netflix/Prime-style; VOD only, B2C). The workload is read-heavy, bursty, latency-sensitive at playback, and bandwidth-dominated, which makes it CDN-first. This brief states the architecture and operational strategy across all five areas; it is intentionally decision-focused and concise.

1 · Context & Assumptions

- **Clients** — mobile and Smart-TV apps are primary; web is secondary and served as a purely static frontend.
- **Region** — single region, `us-east-1`, by deliberate choice to keep initial operational and financial complexity down.
- **Cloud & lock-in** — everything on AWS; AWS lock-in is accepted by default, and open-source is self-hosted only where it is cheaper than the managed AWS equivalent (most visibly observability).
- **HA pattern** — every control-plane component runs with extra replicas and pod `antiAffinity` spread across nodes and AZs (stated once, implied throughout).
- **Scope** — assumed AWS architecture (§2), observability (§3), alerting & escalation (§4), cost optimization (§5). The media production pipeline (ingest/transcode/packaging/DRM) is the content team's, out of scope.

2 · Assumed AWS Architecture

2.1 Networking & routing. Single VPC (`10.0.0.0/16`) across three AZs in three subnet tiers: **public** (NAT only), **workload-private /20** (EKS nodes, ElastiCache, internal NLB), and **isolated DB** (Aurora subnet group, VPC-endpoint ENIs; no internet route). **One NAT Gateway per AZ** for redundant egress. **No public ALB:** CloudFront fronts both paths — `/*` → S3 static frontend, `/api/*` → API Gateway → VPC Link → internal NLB → nginx ingress → EKS. CloudFront (over Cloudflare) keeps web + API on one domain, removing CORS; API Gateway provides per-key rate-limiting, authorizers, access logs and versioning. VPC endpoints (gateway: S3, DynamoDB; interface: SQS, SNS, KMS, ECR, STS) keep AWS-service traffic off the NAT.

2.2 Compute — two EKS clusters. A `prod` cluster for workloads and a dedicated `ops` cluster in a **separate AWS account** (PrivateLink) for the tooling that operates them — observability, ArgoCD (GitOps for both), synthetic probes, and self-hosted GitHub Actions runners — with a dead-man's-switch **heartbeat to PagerDuty**. `prod` nodegroups: `system`, `app-stateless` (Spot + Graviton), `app-stateful`; `ops`: `platform`, `ci` (Spot, scale-to-zero). ECS was evaluated and dropped (constrained on autoscaling, events, observability). Failure-boundary guardrails: `ResourceQuota/LimitRange`, default-deny `NetworkPolicy`, namespace RBAC + IRSA, PodDisruptionBudgets, a staging mirror cluster, an add-on admission process, etcd hygiene, and Cluster Autoscaler (Karpenter not adopted — no team operating experience).

2.3 Service mesh. Istio in **ambient mode**: per-node ztunnel for L4 mTLS, with a Waypoint Proxy enabling L7 features per workload as needed — no sidecar per pod.

2.4 Ingress. nginx ingress behind the internal NLB; access logs in JSON shipped directly to the OpenTelemetry Collector (no broker, no custom metrics app).

2.5 Autoscaling. KEDA scales on SQS queue depth, custom metrics and DB queries, including scale-to-zero for spiky workers.

2.6 Authentication. Cognito with federated login (Meta, Google, Apple); Cognito groups map to IAM policies; OIDC/SAML preserves an IdP-migration path.

2.7 Secrets & config. Secrets Manager (sensitive credentials, native RDS rotation), SSM Parameter Store SecureString (sensitive config without rotation), ConfigMaps via GitOps (non-sensitive), IRSA (pod → AWS auth with no persisted keys), and External Secrets Operator (syncs Secrets Manager into native k8s Secrets).

2.8 Data layer — polyglot by CAP profile. A store per data class. **CP — Aurora Serverless v2** (PostgreSQL-compatible): billing/subscriptions/payments, identity/auth state, and the canonical catalog (system of record & event source). **AP — DynamoDB:** read catalog, watch history/progress (last-write-wins by client timestamp), recommendations, append-only view events, session state. **ElastiCache Redis** for hot cache and ephemeral session; **S3 + CloudFront** for media (immutable after publish); **catalog search via Aurora tsvector**. Read models are fed from Aurora to DynamoDB via SNS + SQS, decoupling reads from the writer.

2.9 Messaging, events & email. **SNS + SQS** is the only messaging backbone. Every publishing service uses the **transactional outbox** via a polling worker: the business row and an `outbox` row commit in one transaction; a worker reads unsent rows (`SELECT ... FOR UPDATE SKIP LOCKED`), publishes to SNS, marks them sent, and loops every 1–2 s; consumers (via SQS) are idempotent. **SES** handles transactional email, with delivery events flowing SNS → SQS → a handler that maintains the bounce list and reputation.

2.10 Media delivery. Video in S3, delivered via CloudFront with per-session signed cookies; the web app covers catalog, player and the authorization flow.

2.11 Perimeter security. WAF v2 on CloudFront (AWS Managed Rules — OWASP, Known Bad Inputs, IP Reputation — plus 2000 req/5 min per-IP rate limiting); Shield Standard for volumetric DDoS; GuardDuty account-wide, findings routed SNS → SQS → a severity-classifying handler into the alerting pipeline.

2.12 DNS. Route 53 alias records to the CloudFront distribution; TLS via ACM in `us-east-1`, auto-renewed.

2.13 Backup & recovery. Aurora continuous PITR + daily snapshots (30 d); DynamoDB PITR on every table (35 d); critical S3 buckets versioned with tag-driven lifecycle; Secrets Manager version rollback (30 d); EKS resources rebuilt from Terraform + ArgoCD. A **quarterly restore drill** proves the Aurora restore works, timed and run from the runbook.

2.14 Infrastructure as Code. Terraform (over CDK); state in per-environment S3 with KMS CMK and a DynamoDB lock; mono-repo (`platform/`, `services/<name>/`, per-env dirs, shared modules). PR workflow: Atlantis (`plan/fmt/tflint`) → Infracost cost-diff in the PR → review → `apply` staging → `apply` prod (second approval). Mandatory tagging is enforced (untagged resources fail the plan); weekly drift detection opens an issue with the diff.

3 · Monitoring & Observability

3.1 Stack. Self-hosted in the `ops` cluster: **VictoriaMetrics** (metrics), **Grafana Tempo** (traces), **Grafana Loki** (logs) — all S3-backed — visualized in **Grafana OSS**, ingested through a single **OpenTelemetry Collector** (`prod`'s collector forwards to `ops` over PrivateLink). No Grafana Cloud.

3.2 Ingestion pipeline. The collector reads JSON nginx logs (`filelog`), extracts fields (`transform`), derives metrics (`logstometrics`), and exports in parallel to Loki, VictoriaMetrics (`remote_write`) and Tempo (OTLP). **Tail sampling:** 100% of error, over-SLO-p99 and synthetic-probe traces; 1% of normal traffic.

3.3 Governance. OIDC SSO; RBAC per folder/org; dashboards versioned as YAML via the Grafana Operator (no free editing in prod); alert rules versioned in the same repo.

3.4 Lock-in. Open-source Grafana stack, no subscription; portable off AWS except for S3 (commodity object storage).

3.5 SLOs & error budget. Seven SLOs centred on **user journeys**, on a 28-day window, measured while the region is available:

Journey	SLO	Journey / SLO
Login <code>/auth/login</code> < 500 ms	99.9%	Checkout / billing — 99.95%
Playback start (manifest < 2 s)	99.5%	Catalog search <code>/search</code> < 500 ms — 99.5%
Continuous playback (segment < 1 s)	99.9%	Transactional email < 5 min — 99.5%
Playback quality (no rebuffer, client-reported)	99.5%	Error budget = (1 – SLO)

Multi-window burn-rate alerts give objective severity: 1 h @ 14.4× → P0/P1 page; 6 h @ 6× → P1 page; 24 h @ 3× → P2 notify; 3 d @ 1× → P3 backlog. **Error-budget policy:** >50% normal deploy; 10–50% extra review; <10% feature freeze; exhausted → halt + postmortem.

3.6 Synthetic probes. Prometheus Blackbox Exporter for single-step checks (TLS expiry, health, manifest fetch, CloudFront edge, DNS, search); canary **CronJobs** in the `ops` cluster for multi-step journeys (full login; playback start; continuous playback with byte validation; sandbox checkout; transactional email via IMAP). Escalation suppresses false positives by consecutive failures + corroboration with the real-traffic SLI. **Dual SLI:** real traffic is canonical, probes are the always-available early warning. Probes are isolated (minimal rotating IAM, dedicated IDs/mailbox, PSP test mode).

4 · Alerting & Escalation

4.1 Severity. Severity is impact + budget consumption, not a feeling:

Sev	Definition	Examples
P0	Platform unavailable / revenue stopping	Global playback broken; auth down; checkout down
P1	Critical functionality degraded, SLO at risk	Regional playback < 95%; p95 > 2 s; email bounce > 5%
P2	Significant degradation w/ workaround, or early P1	Cache hit dropping; 5xx < 1%; transcoding time rising
P3	Trend warning — no page	Disk slowly rising; cost out of band

4.2 Tooling. PagerDuty for paging and escalation — kept external to the cluster so alerting survives a cluster outage.

4.3 Oncall. Weekly rotation (Monday 9 AM handover), 24/7 primary + secondary, minimum 4 engineers, 15-minute ritualized handover, stipend + comp day. A separate security schedule pages the security engineer (manager fallback), independent of the operational rotation.

4.4 Escalation by severity. **P0** — ack 5 min, then secondary (10) → eng manager (20) → director/CTO (40); war room via PagerDuty + Slack. **P1** — ack 10 min, then secondary (15) → manager (30). **P2** — business hours, ack 1 h. **P3** — no page; Jira ticket via webhook. A security P0 escalates in parallel to security, legal/comms and the CTO.

4.5 Runbooks. Every alert carries a runbook (`runbook_url` in the payload), versioned and PR-reviewed as code; template = symptom, likely causes, copy-paste diagnosis, risk-ordered remediation, escalation criteria, post-incident link. Rules: no alert ships without a runbook; each incident generates a runbook-review item; an outdated runbook is an operations bug.

4.6 Postmortems. Every P0/P1 produces a blameless postmortem from a Git template (timeline, measured impact, root cause, contributing factors, action items with owner/deadline); draft in 5 business days, finalized in 10; a bi-weekly meeting reviews them and tracks action items.

5 · Cost Optimization

5.1 Visibility (first). Mandatory tagging enforced in Terraform (`Environment`, `Service`, `Owner`, `CostCenter`, `ManagedBy`, `Lifecycle`; untagged fails the plan), feeding Cost Explorer, CUR-in-S3 (Athena), Budgets, Cost Anomaly Detection → Slack, Grafana cost dashboards (cost beside CPU), and Cloudcraft (per-component cost on a live diagram).

5.2 Optimization (by impact). **Compute (≈40–60%)** — Cluster Autoscaler tuning, Spot on stateless nodegroups, Graviton/ARM64, VPA in recommender mode (PR-reviewed), scheduled non-prod shutdown nights/weekends. **Storage (≈10–15%)** — S3 Intelligent-Tiering, lifecycle (IA → Glacier → Deep Archive), `gp3` over `gp2`, orphan-snapshot sweeps, enforced CloudWatch Logs retention. **Orphans** — weekly sweep for zero-target load balancers, unassociated EIPs, idle NATs, stopped RDS. **Data transfer (≈5–15%)** — VPC gateway/interface endpoints remove the \$0.045/GB NAT pass-through; Istio topology-aware routing cuts cross-AZ traffic. **Database** — Aurora ACU tuning, RDS Proxy, DynamoDB on-demand vs provisioned per table, TTL on ephemeral data. **Reserved capacity** — Savings Plans applied last (after right-sizing/Spot/Graviton stabilize), targeting 70–80% coverage.

5.3 Continuous operation. Daily anomaly detection → Slack; weekly automated cost report (top costs, top changes, top recommendations); monthly orphan cleanup and Savings Plan adjustment; quarterly architectural review of any disproportionately rising layer.